

SKILL 20

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <string.h>

typedef struct {
    int job_id;
    pid_t pid;
    pid_t pgid;
    char status[20];
} Job;

Job job;

/* Child handles SIGTSTP */
void stop_handler(int sig) {
    printf("\nChild received SIGTSTP\n");
    printf("Child is being suspended\n");
    fflush(stdout);

    raise(SIGSTOP);
}
```

```
void show_job() {
    printf("Job ID: %d | PID: %d | PGID: %d | Status: %s\n",
        job.job_id,
        job.pid,
        job.pgid,
        job.status);
}
```

```

int main() {
    int status;

    printf("=== Task 20: Process Groups and Job Suspension ===\n");

    /* Prevent parent from being stopped during terminal control changes */
    signal(SIGTTOU, SIG_IGN);

    pid_t shell_pgid = getpgrp();

    printf("\nShell Process Group ID: %d\n", shell_pgid);

    printf("\n=== Create Child Process ===\n");

    pid_t pid = fork();

    if (pid < 0) {
        perror("fork");
        return 1;
    }

    if (pid == 0) {

        /* Child becomes leader of a new process group */
        setpgid(0, 0);

        signal(SIGTSTP, stop_handler);

        printf("Child started\n");
        printf("Child PID: %d\n", getpid());
        printf("Child Process Group ID: %d\n", getpgrp());

        for (int i = 1; i <= 8; i++) {
            printf("Child working... %d\n", i);
            fflush(stdout);
            sleep(1);
        }
    }
}

```

```

        printf("Child completed normally\n");

        exit(0);
    }

    /* Parent assigns child to its own process group */
    setpgid(pid, pid);

    job.job_id = 1;
    job.pid = pid;
    job.pgid = pid;

    strcpy(job.status, "Running");

    printf("\n=== Job Table ===\n");
    show_job();

    printf("\n=== Transfer Terminal Control ===\n");

    if (tcsetpgrp(STDIN_FILENO, job.pgid) == 0)
        printf("Terminal transferred to child process group\n");
    else
        perror("tcsetpgrp");

    sleep(2);

    printf("\n=== Suspend Job ===\n");

```

```

    /* Send SIGTSTP to entire child process group */
    kill(-job.pgid, SIGTSTP);

    waitpid(pid, &status, WUNTRACED);

    if (WIFSTOPPED(status)) {
        strcpy(job.status, "Stopped");

        printf("Child process suspended successfully\n");
    }

    printf("\n=== Updated Job Table ===\n");
    show_job();

    printf("\n=== Restore Terminal Ownership ===\n");

    tcsetpgrp(STDIN_FILENO, shell_pgid);

```

```
printf("Terminal returned to parent process\n");  
printf("\nJob state preserved while stopped\n");  
sleep(2);  
printf("\n=== Resume Stopped Job ===\n");  
kill(-job.pgid, SIGCONT);  
strcpy(job.status, "Running");  
printf("SIGCONT sent to process group\n");  
show_job();  
printf("\n=== Transfer Terminal Again ===\n");  
tcsetpgrp(STDIN_FILENO, job.pgid);  
printf("Terminal transferred back to resumed job\n");  
printf("\nWaiting for resumed job to finish...\n");  
waitpid(pid, &status, 0);  
strcpy(job.status, "Completed");
```

```

    }

    /* Return terminal to parent */
    tcsetpgrp(STDIN_FILENO, shell_pgid);

    printf("\n=== Final Job State ===\n");
    show_job();

    printf("\n=== Terminal Ownership Restored ===\n");
    printf("Terminal returned to shell process group\n");

    printf("\n=== Task 20 Verification ===\n");
    printf("SIGTSTP handled successfully\n");
    printf("Process suspended successfully\n");
    printf("Job state preserved\n");
    printf("Stopped process resumed using SIGCONT\n");
    printf("Process group created\n");
    printf("Process group ID assigned\n");
    printf("Terminal control transferred\n");
    printf("Terminal ownership restored\n");
    printf("Signal coordination completed\n");

    printf("\n=== Task 20 Complete ===\n");

    return 0;
}

```

Output:

```
=== Task 20: Process Groups and Job Suspension ===
```

```
Shell Process Group ID: 658
```

```
=== Create Child Process ===
```

```
=== Job Table ===
```

```
Child started
```

```
Job ID: 1 | PID: 659 | PGID: 659 | Status: Running
```

```
Child PID: 659
```

```
=== Transfer Terminal Control ===
```

```
Child Process Group ID: 659
```

```
Terminal transferred to child process group
```

```
Child working... 1
```

```
Child working... 2
```

```
=== Suspend Job ===
```

```
Child received SIGTSTP
```

```
Child is being suspended
```

```
Child process suspended successfully
```

```
=== Updated Job Table ===
```

```
Job ID: 1 | PID: 659 | PGID: 659 | Status: Stopped
```

```
=== Restore Terminal Ownership ===  
Terminal returned to parent process  
  
Job state preserved while stopped  
  
=== Resume Stopped Job ===  
SIGCONT sent to process group  
Job ID: 1 | PID: 659 | PGID: 659 | Status: Running  
Child working... 3  
  
=== Transfer Terminal Again ===  
Terminal transferred back to resumed job  
  
Waiting for resumed job to finish...  
Child working... 4  
Child working... 5  
Child working... 6  
Child working... 7  
Child working... 8  
Child completed normally
```

```
=== Final Job State ===  
Job ID: 1 | PID: 659 | PGID: 659 | Status: Completed  
  
=== Terminal Ownership Restored ===  
Terminal returned to shell process group  
  
=== Task 20 Verification ===  
SIGTSTP handled successfully  
Process suspended successfully  
Job state preserved  
Stopped process resumed using SIGCONT  
Process group created  
Process group ID assigned  
Terminal control transferred  
Terminal ownership restored  
Signal coordination completed
```